C PROGRAMMING TEACHING NOTES

Arrays, Strings and File Handling

Purpose: Detailed, simple-English teaching notes for secondary school beginners, based on the supplied 16-page C Programming document.

Source note: The supplied PDF is scanned and several pages contain repeated/overlapping material. The notes follow the document's sequence and explain repeated material once in full.

CONTENTS

1. One-Dimensional Arrays
2. Two-Dimensional Arrays
3. Strings and String Handling
4. Exercise 4.8
5. File Handling
6. Exercise 4.9
7. Completeness Check
8. Final Teacher Revision Guide

1. ONE-DIMENSIONAL ARRAYS

Simple meaning

An array is a group of values stored under one variable name. The values normally have the same data type. Instead of creating many separate variables, one array can store many values.

Declaration

```
int numbers[5];
```

int = data type. **numbers** = array name. **[5]** = five elements.

Initialisation

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Indexes are 0, 1, 2, 3 and 4. Therefore `numbers[0]` is 10 and `numbers[4]` is 50.

Accessing and changing

`printf("%d", numbers[0]);` prints the first element.

`numbers[2] = 100;` changes the third element to 100.

Program Example 4.15

```
#include <stdio.h>
int main()
{
int numbers[5] = {1, 2, 3, 4, 5};
printf("First element: %d\n", numbers[0]);
printf("Second element: %d\n", numbers[1]);
numbers[3] = 10;
printf("Changed fourth element: %d\n", numbers[3]);
return 0;
}
```

The program creates an array, prints the first and second elements, changes the fourth element, and prints the changed value.

How to explain it to students

“Students, imagine five boxes in one row. Each box stores one number. The boxes have positions called indexes. In C, the first box is number 0, so five boxes have indexes 0 to 4.”

Common mistakes

Students may think the first index is 1. They may also use `numbers[5]` for the fifth element of a five-element array. The last valid index is 4.

Key points

1. An array stores several values.
2. Values normally have the same data type.
3. An index identifies a position.
4. Indexing starts at 0.
5. Elements can be accessed and changed.

2. TWO-DIMENSIONAL ARRAYS (MATRICES)

Simple meaning

A two-dimensional array is like a table with rows and columns. For example, student marks can be organised in rows and columns.

Declaration

```
int matrix[3][3];
```

The first 3 represents rows and the second 3 represents columns. A 3 × 3 matrix has 9 elements.

Initialisation

```
int matrix[2][2] = {{1, 2}, {3, 4}};
```

Think of it as:
Row 0: 1 2
Row 1: 3 4

Accessing

A two-dimensional array uses two indexes: `matrix[row][column]`.

`matrix[0][0]` is 1. `matrix[1][1]` is 4.

Changing

`matrix[1][0] = 100;` changes the value at row 1, column 0.

Program Example 4.16

```
#include <stdio.h>
int main()
{
int matrix[2][2] = {{1, 2}, {3, 4}};
printf("Element at row 1, column 1: %d\n", matrix[0][0]);
printf("Element at row 2, column 2: %d\n", matrix[1][1]);
matrix[1][0] = 100;
printf("Changed element at row 2, column 1: %d\n", matrix[1][0]);
return 0;
}
```

How to explain it to students

“A one-dimensional array is like one row of boxes. A two-dimensional array is like a table. To find a value, we need both the row and the column.”

Key points

1. A 2D array has rows and columns.
2. It is similar to a table or matrix.
3. Two indexes are used.
4. The first index identifies the row.
5. The second identifies the column.

3. STRINGS AND STRING HANDLING

What is a string?

A string is text made from characters. In C, a string is stored as an array of characters.

Declaration

```
char name[10];
```

char stores character data. The array needs enough space for the text and the null character.

Null character

C uses `\0` to mark the end of a string. “John” is stored conceptually as J, o, h, n, \0.

Initialisation

```
char name[] = "John";
```

gets()

The document introduces `gets()` for reading a line, but warns that it is normally avoided because it can cause buffer overflow.

fgets()

`fgets(name, sizeof(name), stdin);` reads text while considering the available storage size.

puts()

`puts("Hello, World!");` displays a string.

strlen()

`strlen(name)` returns the number of characters in a string, not counting the null character. “John” has length 4.

Program Example 4.18

The document uses the string `"Mwananzengo"`, prints it with `puts()`, and uses `strlen()`. The shown output is length 11.

How to explain it to students

“Students, a string is simply text. A word is made of characters, and C keeps those characters in a character array. The special \0 tells the computer where the string ends.”

Key points

1. A string stores text.
2. A string is an array of characters.
3. \0 marks the end.
4. puts() displays a string.
5. strlen() counts characters without counting \0.

4. EXERCISE 4.8 — ARRAYS AND STRINGS

Question 1 — Array error

The scanned question shows an array declared with five spaces but six initial values, and a loop boundary that must be checked. A corrected form is:

```
int numbers[6] = {2, 3, 4, 6, 8, 10};
for (int i = 0; i < 6; i++)
{ printf("%d ", numbers[i]); }
```

Question 2 — Sum of five numbers

```
int numbers[5] = {1,2,3,4,5};
int sum = 0;
for (int i = 0; i < 5; i++)
sum = sum + numbers[i];
printf("Sum = %d\n", sum);
```

The sum is 15.

Question 3 — Reverse array

```
for (int i = 4; i >= 0; i--)
printf("%d ", arr[i]);
```

For {1,2,3,4,5}, output is 5 4 3 2 1.

Question 4 — Full name input

The document marks `scanf("%s", name);` as the problem because %s stops at whitespace. The lesson earlier provides `fgets(name, sizeof(name), stdin);` as a better approach.

Question 5 — Count characters without strlen()

Move through the string until `\0`:

```
int count = 0;
while (word[count] != '\0')
count++;
```

Question 6 — puts() and printf()

`puts()` mainly displays a string. `printf()` is more flexible and supports formatted output.

5. FILE HANDLING

Introduction

File handling allows a C program to store and retrieve information from files rather than keeping all information only on the console.

Why file handling is important

Permanent storage: data remains after the program stops.
Handling large data: files can store large amounts of information.
Sharing data: files can help transfer information between programs.

Basic functions

`fopen()` — opens a file.
`fclose()` — closes a file.
`fprintf()` — writes data to a file.
`fscanf()` — reads data from a file.

Opening a file

```
FILE *file;
file = fopen("myfile.txt", "w");
```

The filename is myfile.txt and the second argument is the mode.

File modes

r = read; file must exist.
w = write; creates or overwrites.
a = append; adds to the end.
r+ = read and write.
w+ = write and read.
a+ = append and read.

Program Example 4.19 — Writing

```
FILE *file = fopen("myfile.txt", "w");
if (file == NULL)
{
printf("Error in opening a file!\n");
return 1;
}
fprintf(file, "Hello, this is a file handling example!\n");
fprintf(file, "File handling makes data storage easy.\n");
fclose(file);
```

The program opens the file, checks for failure, writes two lines and closes it.

Program Example 4.20 — Reading

The document demonstrates opening myfile.txt in read mode and using `fscanf()` to read one word into a character array, then printing it.

Simple teaching sequence

OPEN → READ or WRITE → CLOSE

Writing: `fopen()` → `fprintf()` → `fclose()`
Reading: `fopen()` → `fscanf()` → `fclose()`

6. EXERCISE 4.9 — FILE HANDLING

Question 1 — Wrong file mode

If the program opens a file using `"r"` and then tries to write with `fprintf()`, the mode is wrong. Use `"w"` when writing is intended.

Question 2 — Create info.txt

```
FILE *file = fopen("info.txt", "w");
fprintf(file, "Learning C is fun!\n");
fprintf(file, "File handling makes programming easier.\n");
fclose(file);
```

Question 3 — Read message.txt

Open the file with `"r"`, check for `NULL`, read the content using the technique taught, print it, then close the file.

Question 4 — Error when reading

Always check whether `fopen()` successfully opened the file before using `fscanf()`.

Question 5 — Append instead of overwrite

Change `"w"` to `"a"`. Append mode adds new text to the end.

Question 6 — Count characters in story.txt

This requires reading characters and counting them. A straightforward C approach uses `fgetc()` until `EOF`. These are additional concepts and are not explained in the supplied pages.

Question 7 — Buffer size

For `char text[100]`, unrestricted `%s` may allow too much input. A safer form is `%99s`, leaving space for the ending `\0`.

7. COMPLETENESS CHECK

The supplied 16-page scan was checked from the first page to the last. The major topics, examples, tables and exercises are covered in these notes. Several pages repeat earlier material, so repeated scans were identified rather than treated as new concepts.

Topic	Status
One-dimensional arrays	Explained
Array declaration / initialisation	Explained
Array indexing / access	Explained
Examples 4.15 and 4.16	Explained
Strings and string functions	Explained
Examples 4.17 and 4.18	Explained
Exercise 4.8	Covered
File handling and file modes	Explained
Examples 4.19 and 4.20	Explained
Exercise 4.9	Covered
Repeated/overlapping scan pages	Checked

8. FINAL TEACHER REVISION GUIDE

Teach the material in this order:

1. Review ordinary variables.

2. Introduce one-dimensional arrays and indexing.

3. Teach declaration, initialisation, access and changing values.

4. Practise Example 4.15.

5. Introduce two-dimensional arrays as tables.

6. Teach rows and columns and practise Example 4.16.

7. Introduce characters and strings.

8. Explain the null character \0.

9. Teach gets(), fgets(), puts() and strlen().

10. Work through Exercise 4.8.

11. Introduce files and permanent storage.

12. Teach fopen() and file modes.

13. Teach writing with fprintf().

14. Teach reading with fscanf().

15. Teach closing with fclose().

16. Work through Examples 4.19 and 4.20.

17. Finish with Exercise 4.9.

IMPORTANT TERMS

Array — a collection of values under one name.
Element — one value inside an array.
Index — the position of an element.
String — text stored as characters.
Null character — \0, marking the end of a string.
File — stored information that can be used later.
Append — add information to the end of a file.

IMPORTANT SYNTAX

```
int numbers[5];
int numbers[5] = {1,2,3,4,5};
numbers[0];
int matrix[2][2];
char name[] = "John";
strlen(name);
FILE *file;
file = fopen("data.txt", "r");
fprintf(file, "Hello");
fscanf(file, "%s", text);
fclose(file);
```

FINAL REMINDER FOR THE TEACHER

The most important beginner foundation is array indexing. Make sure students understand why the first element is [0]. For file handling, teach the simple pattern **open** → **read/write** → **close** before moving to complete programs.